

# **南方硅谷 SSV6358 基于 BlueZ 的 BLE 使用说明**

## 简介

文档主要介绍 6358 系列芯片 ble 功能的使用介绍。

### 发布说明:

| 版本   | 日期       | 作者        | 描述 |
|------|----------|-----------|----|
| V1.0 | 20230306 | icommsemi | 初版 |
|      |          |           |    |
|      |          |           |    |

## 目录

|                                |    |
|--------------------------------|----|
| 1. 驱动编译和加载 .....               | 4  |
| 1.1 内核蓝牙协议栈配置 .....            | 4  |
| 1.2 基于 USB 或 SDIO 接口 .....     | 4  |
| 1.3 基于 UART 接口 .....           | 6  |
| 2. 配置文件中 BLE 设置说明 .....        | 7  |
| 2.1 BLE 的 MAC 地址设置 .....       | 7  |
| 2.2 进入 BLE DTM 测试模式 .....      | 8  |
| 3. Bluez 测试 BLE 基本功能说明 .....   | 8  |
| 3.1 作为 peripheral 测试广播连接 ..... | 8  |
| 3.2 作为 central 测试扫描连接 .....    | 10 |

## 1. 驱动编译和加载

### 1.1 内核蓝牙协议栈配置

使用 Bluez 协议栈需要 Linux kernel 的 bluetooth feature 支持，需要在 kernel\_menuconfig 或 menuconfig 选上对应的配置,不同芯片提供的内核配置会有一些差异，大体配置相同，以上提供参考：

#### 1) 配置使能内核部分蓝牙功能：make kernel\_menuconfig

```
[*] Networking support --->
    <*> Bluetooth subsystem support --->
        [*] Bluetooth Low Energy (LE) features
```

#### 2) 配置蓝牙协议支持 UART 接口设备：make kernel\_menuconfig

```
[*] Networking support --->
    <*> Bluetooth subsystem support --->
        Bluetooth device drivers --->
            <*> HCI UART driver
                [*] UART(H4) protocol support
```

#### 3) 建议验证及使用驱动 BLE feature 的内核版本及 Bluez 的版本

Linux kernel >= v4.4

Bluez >= v5.50

### 1.2 基于 USB 或 SDIO 接口

#### 1) 驱动编译选项修改 config.mak

```
## BLE HCI usage (注意与NIMBLE的使用为2选1)
CONFIG_BLE ?= y
ccflags-$(CONFIG_BLE) += -DCONFIG_BLE

# BLE HCI BUS 取值：
# CONFIG_BLE_HCI_OVER_UART:0
# CONFIG_BLE_HCI_OVER_HWIF:1 USB or SDIO, base on HWIF
ccflags-y += -DCONFIG_BLE_HCI_BUS=1
```

#### 2) 配置文件修改 firmware image 路径及加载 bin 文件名称，使用 ble 功能需使用名称为 ssv6x5x-sw\_ble.bin 的文件

```
#####
# Firmware setting
#
# Priority.1 insmod parameter "cfgfirmwarepath" (insmod的時候帶入)
# Priority.2 firmware_path (在下面設定)
# Priority.3 default firmware (前兩個都沒設定會load firmware header)
#
#####
#firmware_path = /home/xxx/xxx/image/
#firmware_path = /home/icom/Downloads/6358/ssv6x5x/image/
#####
# Firmware name
#
# 若有設定firmware path的話才有用，內部預設firmware name使用"ssv6x5x-sw.bin"
#
#####
#firmware_name = ssv6x5x-sw.bin
#firmware_name = ssv6x5x-sw_ble.bin
```

### 3) 执行 load.sh 加载驱动后通过 kernel 中 log 信息确认 BLE HCI 功能，USB 或 SDIO 接口

```
[62944.408419] ble_hci_init
[62944.408420] cfg.ble_dtm 0
[62944.408425] hci device name:SSV6XXX_USB
[62944.408593] \x1b[32mssv ble hci open():78 \x1b[0m
[62944.408595] Config BLE HCI over HWIF
[62944.409102] hci device id:1
```

### 4) 测试 HCI 应用接口：

以下使用 USB 接口接入基于 ubuntu 系统，该说明使用基于 Ubuntu 22.04, BlueZ 5.64 版本测试

#### ➤ 确认加载 HCI 设备成功

sudo hciconfig

```
icom@icom-virtual-machine:~/Downloads/6358/ssv6x5x$ hciconfig
hci1: Type: Primary Bus: USB
      BD Address: 14:B2:E5:FF:AA:E9 ACL MTU: 27:8 SCO MTU: 0:0
      DOWN
      RX bytes:0 acl:0 sco:0 events:14 errors:0
      TX bytes:0 acl:0 sco:0 commands:14 errors:0
```

#### ➤ 启动 HCI 设备开始工作

sudo hciconfig hci1 up

```
icom@icom-virtual-machine:~/Downloads/6358/ssv6x5x$ sudo hciconfig hci1 up
icom@icom-virtual-machine:~/Downloads/6358/ssv6x5x$ hciconfig
hci1: Type: Primary Bus: USB
      BD Address: 14:B2:E5:FF:AA:E9 ACL MTU: 27:8 SCO MTU: 0:0
      UP RUNNING
      RX bytes:0 acl:0 sco:0 events:30 errors:0
      TX bytes:0 acl:0 sco:0 commands:30 errors:0
```

## 1.3 基于 UART 接口

### 1) 驱动编译选项 config.mak,修改

```

} ## BLE HCI usage (注意与NIMBLE的使用为2选1)
| CONFIG_BLE ?= y
| ccflags-$(CONFIG_BLE) += -DCONFIG_BLE
|
| # BLE HCI BUS 取值:
| # CONFIG_BLE_HCI_OVER_UART:0
| # CONFIG_BLE_HCI_OVER_HWIF:1 USB or SDIO, base on HWIF
| ccflags-y += -DCONFIG_BLE_HCI_BUS=0

```

### 2) 配置文件修改 firmware image 路径及加载 bin 文件名称, 使用 ble 功能需使用名称为 ssv6x5x-sw\_ble.bin 的文件

```

#####
# Firmware setting
#
# Priority.1 insmod parameter "cfgfirmwarepath" (insmod的時候帶入)
# Priority.2 firmware_path (在下面設定)
# Priority.3 default firmware (前兩個都沒設定會load firmware header)
#
#####
#firmware_path = /home/xxx/xxxx/image/
firmware_path = /home/icom/Downloads/6358/ssv6x5x/image/
#####
# Firmware name
#
# 若有設定firmware path的話才有用, 內部預設firmware name使用"ssv6x5x-sw.bin"
#
#####
#firmware_name = ssv6x5x-sw.bin
firmware_name = ssv6x5x-sw_ble.bin

```

### 3) 执行 load.sh 加载驱动后通过 kernel 中 log 信息确认 BLE HCI UART 功能

```

[57588.424058] ieee80211 phy4: New interface create p2p0
[57588.424071] SSV host driver version 1400, Build Data 2023-02-22
[57588.432421] SSV firmware version 10947
[57588.432425] ble_hci_init
[57588.432426] Config BLE HCI over UART
[57588.432472] ~~~~~INIT SSV CTL GENERIC NETLINK MODULE
[57588.432479] ~~~~~INIT SSV WPAS CTL GENERIC NETLINK MODULE
[57588.432485] SSV6X5X of iComm-semi
[57588.432485] ssv_custom_modify_rf_conf_table

```

### 4) 硬件接口:

PIN30- GPIO 36 - UART2-RXD

PIN31- GPIO 37 - UART2-TXD

### 5) 串口参数: 115200-8N1

### 6) 测试 HCI uart 应用接口:

以下例子使用 USB 接口, 串口 txrx 通过电平转换接入, 基于 ubuntu 系统, 检测到串口设备为/dev/ttyUSB0, 该说明使用基于 Ubuntu 22.04, BlueZ 5.64 版本测试

- 挂载 HCI 设备，波特率为 115200

```
sudo hciattach -s 115200 /dev/ttyUSB0 any 115200 noflow nosleep
```

- 查看加载的 HCI 设备号

```
sudo hciconfig
```

```
iComm@iComm-virtual-machine:~/Downloads/6358/ssv6x5x$ hciconfig
hci1:  Type: Primary  Bus: UART
       BD Address: 00:00:00:00:00:00  ACL MTU: 0:0  SCO MTU: 0:0
       DOWN
       RX bytes:0 acl:0 sco:0 events:0 errors:0
       TX bytes:0 acl:0 sco:0 commands:0 errors:0
```

- 启动 HCI 设备开始工作

```
sudo hciconfig hci1 up
```

```
iComm@iComm-virtual-machine:~/Downloads/6358/ssv6x5x$ sudo hciconfig hci1 up
iComm@iComm-virtual-machine:~/Downloads/6358/ssv6x5x$ hciconfig
hci1:  Type: Primary  Bus: UART
       BD Address: 14:B2:E5:FF:AA:E9  ACL MTU: 27:8  SCO MTU: 0:0
       UP RUNNING
       RX bytes:200 acl:0 sco:0 events:13 errors:0
       TX bytes:68 acl:0 sco:0 commands:13 errors:0
```

## 2. 配置文件中 BLE 设置说明

### 2.1 BLE 的 MAC 地址设置

目前支持使用的是 public address

```
#####
# MAC address setting
#
# 設定裝置的MAC address,
# Priority.1 hw_mac (在下面設定)
# Priority.2 efuse存在的MAC address
# Priority.3 default MAC (前兩個都沒設定會用驅動寫死的MAC address)
#
#####
hw_mac = 00:23:45:67:89:20|
```

按优先级支持 3 种方式配置 MAC 地址，默认选择优先级最高（priority 1）

如图所示，配置 hw\_mac = 00:23:45:67:89:20，则此时 BLE MAC 地址为 00:23:45:67:89:21

```
iComm@iComm-virtual-machine:~/Downloads/6358/ssv6x5x$ hciconfig
hci1:  Type: Primary  Bus: USB
       BD Address: 00:23:45:67:89:21  ACL MTU: 27:8  SCO MTU: 0:0
       UP RUNNING
       RX bytes:0 acl:28 sco:0 events:732 errors:0
       TX bytes:0 acl:25 sco:0 commands:70 errors:0
```

若未配置 hw\_mac,则取 efuse 存在的 MAC address, 否则使用默认值

## 2.2 进入 BLE DTM 测试模式

测试模式使能需在配置文件中使能 `ble_dtm = 1` (去掉#),此时重启加载驱动后,进入 BLE DTM 测试模式, 其他功能暂不可用;

```
#####
# BLE Setting
# if ble_replace_scan_win and ble_replace_scan_interval both are not 0
# only apply when stack set full scan to link layer
#####
#ble_replace_scan_win = 0x30
#ble_replace_scan_interval = 0x0140
#ble_dtm = 1
```

- 硬件接口:
  - PIN30- GPIO 36 - UART2-RXD
  - PIN31- GPIO 37 - UART2-TXD
- 串口参数: 115200-8N1

## 3. Bluez 测试 BLE 基本功能说明

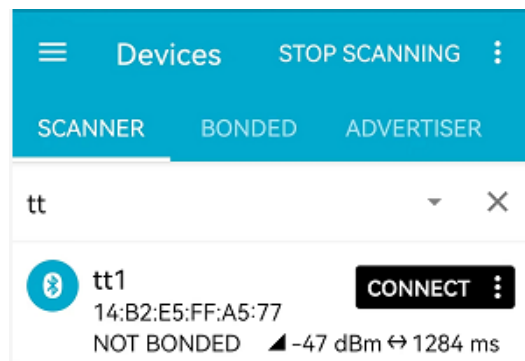
### 3.1 作为 peripheral 测试广播连接

以下使用 bluez 自带的 `bluetoothctl` 工具来测试 (版本: 5.64)

进入 `bluetoothctl` 命令交互界面:

- 在菜单广播设置中, 配置广播名称
  - [Bluetooth]# menu advertise
  - [Bluetooth]# name tt1
  - [Bluetooth]# back
- 返回主菜单页面后打开广播
  - [Bluetooth]# advertise on

此时打开手机 BLE 测试 APP(如 nrfconnect), 可以扫描到设备



默认广播的间隔为 1280ms



## ✓ 如何修改广播间隔

根据不同平台关于 bluetooth 功能配置的不同，这里介绍 3 种方式（在 5.64 版本测试，其他版本需自行检查是否支持特性）

1 ) 配置 /etc/Bluetooth/main.conf 中的广播参数 MinAdvertisementInterval/MaxAdvertisementInterval，应用初始化加载 main.conf 的默认参数

2 ) 应用启动带-E 参数启动实验性特性，通过 bluetoothctl 中 interval 命令可以修改

3 ) 修改/sys/kernel/debug/bluetooth/hcix 中的参数值 adv\_min\_interval/ adv\_max\_interval（需内核支持 debugfs 的 bluetooth 功能）

关于扫描/连接参数的设置方法可以参考上述介绍。

## ➤ 测试 GATT server 连接与读写 characteristic

从主菜单进入 gatt 配置菜单，新增用于测试的 service 和 characteristics

**[Bluetooth]# menu gatt**

**[Bluetooth]# register-service ff01**

提示是否 primary: yes

**[Bluetooth]# register-characteristics ff02 read,write**

提示输入 Value 值:（多字节需带空格，数值为十进制输入）

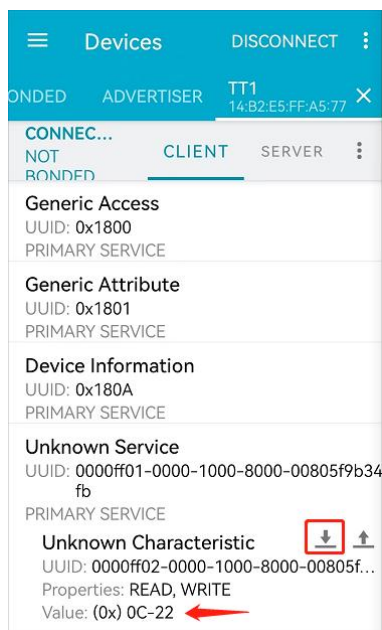
**Enter value 12 34**

**[Bluetooth]# register-application**

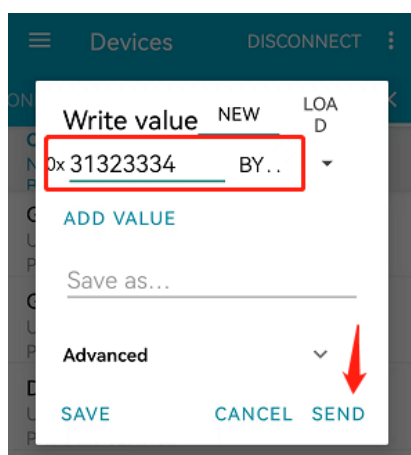
```
[bluetooth]# register-service ff01
[00NEW00] Primary Service (Handle 0x0000)
/org/bluez/app/service0
ff01
[/org/bluez/app/service0] Primary (yes/no): yes
[bluetooth]# register-characteristic ff02 read,write
[00NEW00] Characteristic (Handle 0x0000)
/org/bluez/app/service0/chrc0
ff02
[/org/bluez/app/service0/chrc0] Enter value: 12 34
[bluetooth]# register-application
[00CHG00] Primary Service (Handle 0x0015)
/org/bluez/app/service0
ff01
[00CHG00] Characteristic (Handle 0x0017)
/org/bluez/app/service0/chrc0
ff02
```

此时在进入广播后，打开手机 APP-nrfconnect，扫描到设备后点击[CONNECT],等待连接成功；

如图所示：点击  read value 等于(0x)0C -22



点击  write value: 0x31323334 (BYTE ARRAY),



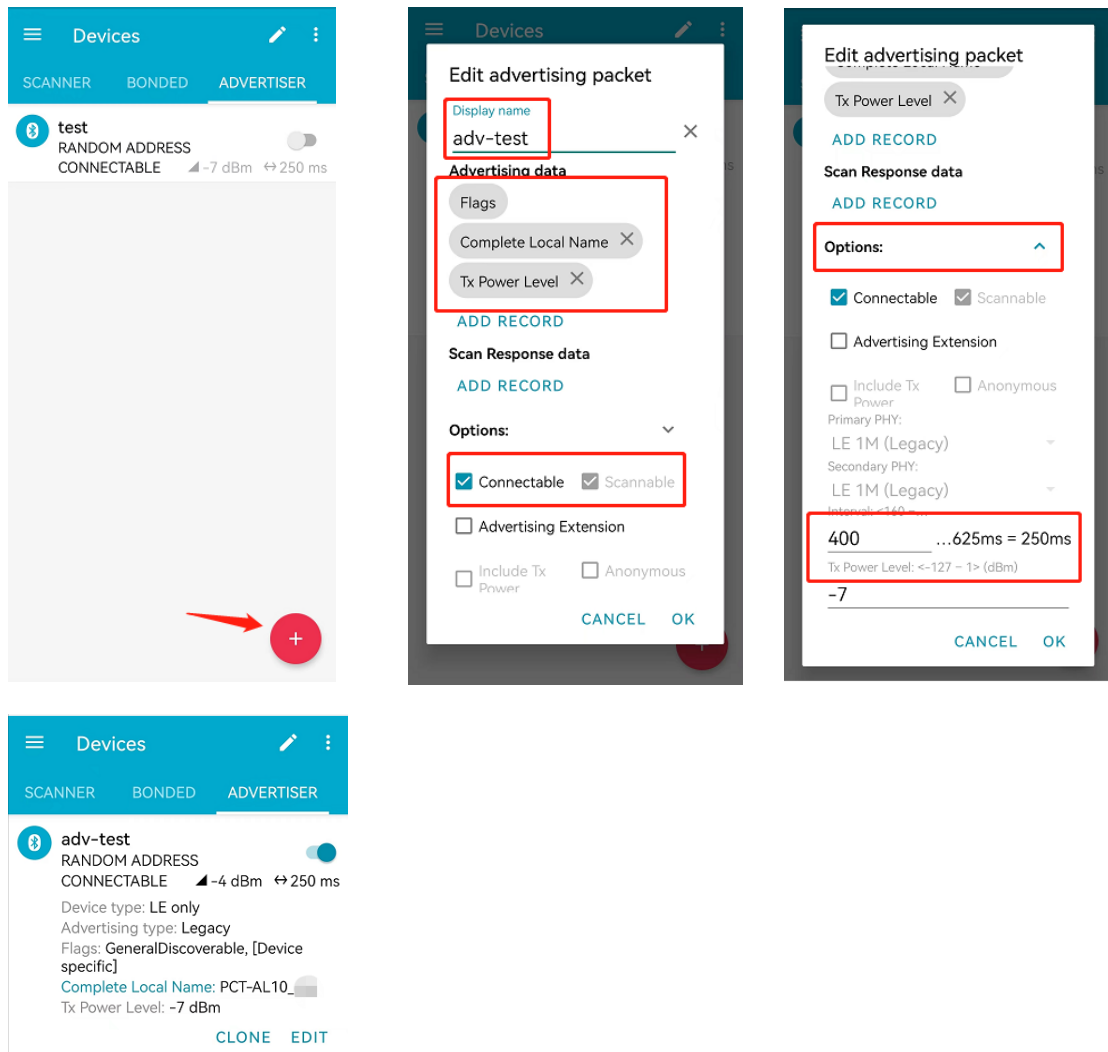
设备侧显示收到数据 3132334

```
[/org/bluez/app/service0/chrc0 ((null))] ReadValue: 30:A1:FA:37:D8:07 offset 0 link LE
[/org/bluez/app/service0/chrc0 ((null))] WriteValue: 30:A1:FA:37:D8:07 offset 0 link LE
31 32 33 34 1234
```

## 3.2 作为 central 测试扫描连接

以下使用 bluez 自带的 bluetoothctl 工具来测试 (版本: 5.64)

- 手机 app-nrfconnect 设置发起广播，  
点击创建 advertiser，设置显示 name 及广播数据，选项勾选 connectable/scannable，点击 Options，设置广播间隔后打开广播，如下图所示：



- 进入 Bluetoothctl 命令行主菜单界面发起扫描设备

**[Bluetooth]# scan on**

```
[bluetooth]# scan on
Discovery started
[CHG] Controller 00:23:45:67:89:21 Discovering: yes
[NEW] Device 4E:CA:36:D8:C2:0C 4E-CA-36-D8-C2-0C
[NEW] Device 9C:C1:2D:7C:B9:EE midea
[NEW] Device 41:D5:9A:69:64:FF 41-D5-9A-69-64-FF
[NEW] Device 72:02:C2:DE:3A:E8 72-02-C2-DE-3A-E8
[NEW] Device 67:49:48:5E:B2:08 67-49-48-5E-B2-08
[NEW] Device 54:A4:93:68:E0:D9 54-A4-93-68-E0-D9
[NEW] Device DA:00:03:98:5E:E6 DA-00-03-98-5E-E6
[NEW] Device 69:94:77:14:A3:9F PCT-AL10
[NEW] Device 71:5D:88:C1:A2:39 71-5D-88-C1-A2-39
[NEW] Device 7E:2E:C3:A0:41:1C 7E-2E-C3-A0-41-1C
[NEW] Device 6A:77:06:81:1D:80 6A-77-06-81-1D-80
[CHG] Device 67:49:48:5E:B2:08 RSSI: -82
```

- 连接设备, 需要 scan 到设备才能进行连接操作, 否则返回连接失败
- [Bluetooth]# connect 69:94:77:14:A3:9F**
- 连接设备成功后, 可以同样操作 read/write value, 参考上一节操作
- 断开设备连接

```
[PCT-AL10_ ]# disconnect
Attempting to disconnect from 69:94:77:14:A3:9F
Successful disconnected
```